

ARCHI DRAFT

MASTER PROJECT INSTRUCTIONS FOR ALL AI AGENTS

Purpose: This document is the common operating instruction for every AI agent and team member working on the Archi Draft application. It must be attached to the project together with the Final Implementation-Ready Architecture document.

SOURCE OF TRUTH PRIORITY

1. Master Architecture Document → 2. Master Project Instructions → 3. Existing Project Code → 4. Current Task Request → 5. General AI Assumptions.

Rule: Never override approved architecture using assumptions. If something is unclear, check the architecture and existing code first. Do not silently invent requirements or redesign the system.

PROJECT OBJECTIVE

Build Archi Draft, a secure project workflow platform connecting CLIENT → ADMIN → DRAUGHTSMAN. The MVP manages project creation, submission, review, draughtsman assignment, assignment acceptance, drawing work, version uploads, client review, correction rounds, final approval and completion.

APPROVED MVP SCOPE

Roles: CLIENT, DRAUGHTSMAN, ADMIN only.

Authentication: Email + Password, Email Verification, Password Reset.

Post-MVP: Student Portal, payments, Google Sign-In, Phone OTP, advanced notifications, proof documents and training features.

MANDATORY AGENT WORKFLOW

READ → UNDERSTAND → CHECK → PLAN → IMPLEMENT → TEST → REPORT

Before coding: read the Master Architecture, inspect the current project structure, read relevant existing files, check for existing implementations, then plan minimal changes. Never jump directly into coding.

PROJECT STRUCTURE DISCIPLINE

Follow the approved Flutter structure in the Master Architecture. Do not randomly place files, create duplicate services/models/repositories, or mix UI, business logic, Firebase logic and data models in one large file. Each file must have one clear responsibility.

Example responsibility separation: UI → screens/widgets; Business logic → controllers/providers/state; Database access → repositories; Firebase integration → services; Data structures → models.

SECURITY NON-NEGOTIABLE

Never make Firestore or Storage public. Never use allow read, write: if true. Apply least privilege. Clients access only authorized projects. Draughtsmen access only authorized assignments. Normal users can never modify ownership, assign themselves, promote themselves to Admin or create invalid workflow states.

FIREBASE STORAGE

Secret or random paths are NOT authorization. Storage authorization must use authentication + project authorization + role + assignment status + folder permission, according to the approved architecture.

Role-specific structure should remain conceptually separated: projects/{projectId}/client_uploads, draughtsman_versions and correction_attachments. Do not use broad write permission for the entire project.

Client: read authorized files; write only client upload area. Draughtsman: read only authorized assigned project files; write only draughtsman version area. Replaced or rejected draughtsmen must lose future access according to the final architecture.

WORKFLOW AND STATE MACHINE

Do not invent statuses or transitions. Follow the Master Architecture. Approved conceptual project states include DRAFT, SUBMITTED, WAITING_ASSIGNMENT, WAITING_ACCEPTANCE, IN_PROGRESS, UNDER_CLIENT_REVIEW, COMPLETED and CANCELLED, subject to the exact final architecture.

Invalid transitions must fail. Correction lifecycle is separate from project lifecycle.

CRITICAL ACTIONS

Critical actions include createProject(), updateDraft(), submitProject(), approveProject(), rejectProject(), assignDraughtsman(), acceptAssignment(), rejectAssignment(), startWork(), uploadVersion(), submitDrawing(), requestCorrection(), resolveCorrection(), approveFinalDrawing(), completeProject() and cancelProject().

Before implementing any critical action verify: actor, role, preconditions, current state, validation, database changes, atomicity, resulting state and activity log.

ACTIVITY LOGS

Activity logs are trusted, immutable audit records. Normal users must not create arbitrary official logs, modify them or delete them. Follow the approved server-side architecture for actor identity, trusted log creation and idempotency.

TRANSACTIONS AND ATOMICITY

Use the approved Atomic Operation Matrix. Use transactions when an operation depends on the latest database state, such as correction limits, assignment acceptance/replacement or version numbering. Use batch writes only when appropriate and values do not require concurrent state calculation.

CORRECTIONS AND VERSIONING

Maximum correction rounds for MVP: 3. The fourth correction must be rejected by backend/database logic, not merely hidden in the UI. Keep correction status separate from project status. Preserve drawing version history according to the approved architecture.

ADMIN AUTHORIZATION

Use only the final approved Admin authorization architecture. Public users must never create or modify Admin authorization records or promote themselves. Do not create an alternative Admin system.

DATABASE DISCIPLINE

Before creating a collection, check the Master Architecture and existing implementation. Avoid duplicate collections and random names. Core data should use proper models rather than unstructured maps everywhere.

TESTING REQUIREMENT

Every completed feature must be verified. Test happy path, invalid input, unauthorized user, invalid state, Firebase/network failures and duplicate actions where relevant.

Security tests must include: cross-client project access, unassigned project access, replaced draughtsman access, ownership modification, Admin privilege escalation, unauthorized file access, invalid state transitions and a fourth correction attempt.

MULTI-AGENT COLLABORATION

No AI agent owns the architecture. The Master Architecture owns the architecture. Before modifying another agent's work: read existing code, understand why it exists, check architecture, and make minimal necessary changes. Do not delete or rewrite unrelated code.

MULTI-AGENT HANDOFF FORMAT

Every agent completing work must report:

1. Task Completed
2. Files Created
3. Files Modified
4. Architecture Compliance
5. Security Impact
6. Testing Performed
7. Next Dependency

DEPENDENCY AND COST DISCIPLINE

Do not add packages unless necessary. Check whether Flutter/Firebase already provides the capability. Prefer maintained, compatible dependencies. Prioritize free tiers and low-cost infrastructure, but never promise a guaranteed \$0 cost.

ARCHITECTURE CHANGE PROTOCOL

No agent may silently change user roles, database architecture, Firebase authorization, Storage authorization, project state machine, assignment system, correction limit or critical workflow. If a change is necessary: STOP → explain the problem → explain why the current architecture cannot support it → propose the smallest change → wait for approval.

DEVELOPMENT ORDER

Phase 0: Project Foundation (Flutter project, structure, dependencies, Firebase configuration).

Phase 1: Authentication.

Phase 2: User Profile.

Phase 3: Firebase Security Foundation.

Phase 4: Client Project System.

Phase 5: Admin Project Management.

Phase 6: Draughtsman Assignment.

Phase 7: Draughtsman Workflow.

Phase 8: Client Review and Corrections.

Phase 9: Project Completion.

Phase 10: Security Audit.

Phase 11: UI Polish.

ABSOLUTELY PROHIBITED

Do not invent features, silently change architecture, duplicate services/models/collections, make Firestore or Storage public, trust UI authorization, trust secret paths, allow broad write access, allow self-promotion, allow invalid state

transitions, ignore existing code, rewrite unrelated code or claim untested code works.

AGENT PRE-TASK CHECKLIST

- ☐ Read Master Architecture
- ☐ Understand current task
- ☐ Check existing files
- ☐ Check existing implementation
- ☐ Avoid duplication
- ☐ Follow approved architecture
- ☐ Do not invent requirements
- ☐ Consider security/database/state impact
- ☐ Plan testing

AGENT COMPLETION CHECKLIST

- ☐ Architecture followed
- ☐ No duplicate functionality
- ☐ Authorization considered
- ☐ Validation implemented
- ☐ Error handling exists
- ☐ State transitions valid
- ☐ Database writes appropriate
- ☐ Storage permissions respected
- ☐ Tests performed
- ☐ Files organized
- ☐ No sensitive data exposed

FINAL AGENT INSTRUCTION

You are implementing one component of an existing approved system. You are NOT independently designing a new application. Always READ → UNDERSTAND → CHECK → PLAN → IMPLEMENT → TEST → REPORT. If your assumptions conflict with the Master Architecture, THE MASTER ARCHITECTURE WINS.